AuditWhy

Why don’t our numbers match?

Seven agents investigate across SEC filings, the industry taxonomy and internal systems, and answer with evidence. Anything without data is struck out, not asserted.

Open Agent Hackathon 2026 · Track: The Agent That Can Explain Why · github.com/Jae3y/AuditWhy

The evidence lives in three systems that disagree

Regulatory filings

SEC EDGAR revenue and the industry code each company reports about itself.

Industry taxonomy

The official SIC hierarchy that groups companies into cohorts.

Internal systems

ERP billing, churn and support escalations, keyed differently again.

Who feels it

A finance or risk lead asks “why did enterprise retention drop?”. Join the systems naively and companies silently fall out. Ask a chatbot and it invents a cause, such as pricing, from data it never looked at.

An investigation, not a chatbot

Ask one question. Get an answer an auditor could sign off.

AuditWhy writes testable hypotheses, queries every source through Zetaris, challenges its own evidence, and reports what it found, what it could not test, and what to do next.

- Verified findings, each citing its source tables
- Claims without data, struck out and listed as untested
- Confidence from statistics, not asserted
- What it learns is remembered for next time

Seven agents that check each other

Scout	Planner	Retriever	**Evaluator**	Analyst	Writer	Reviewer
Guards input, signs the contract	Writes 4 testable hypotheses	Queries Zetaris	Checks data quality, blocks claims	Bounded reasoning chunks	Verified findings only	Enforces caveats, mines memory

Loop 1: data quality

The Evaluator runs its own check. A filer missing from the join sends the Planner back to revise, then it re-checks.

Loop 2: claims

Every claim needs rows behind it. No pricing data, no pricing claim: it moves to untested.

Loop 3: review

The Reviewer rejects a draft until the caveat names every untested factor and statistic.

What’s unique: the Evaluator doesn’t trust the Retriever

3570

IBM’s self-reported SIC code: a summary header, not a leaf

A standard join drops IBM, and its **$62.75B** of revenue with it. The Evaluator runs its own Zetaris data-quality query, catches the orphan, and sends the plan back. Pass 2 reconciles 3570 to 3571 Electronic Computers.

The rule goes into H-MEM. The next question recalls it, skips a whole reasoning stage, and earns the rule more trust. A wrong rule is penalised and superseded.

Nothing is hardcoded: change Oracle’s SIC code to 7370 in the data and the agents catch that orphan too.

Technical stack

Layer	Technology	Role in AuditWhy
Data	Zetaris Lightning SQL, REST	Every query; USL and DQ rule
Build	Meterless Engines	Scout, World Model, Markovian, H-MEM
Power	Meterless Nightshift	Primary model provider
Models	NVIDIA Nemotron	Fast, deep and judge lanes
Service	Python, FastAPI, Pydantic	POST /run, dashboard, CLI
Delivery	Docker, Vercel, GitHub Actions	Clean-clone CI and live demo
Development	Claude Code	Scaffolding, refactors, tests

79 tests and a 7-subsystem conformance suite run on every push. Sample mode runs the full agent loop with no credentials.

How Zetaris and Meterless carry the load

Zetaris

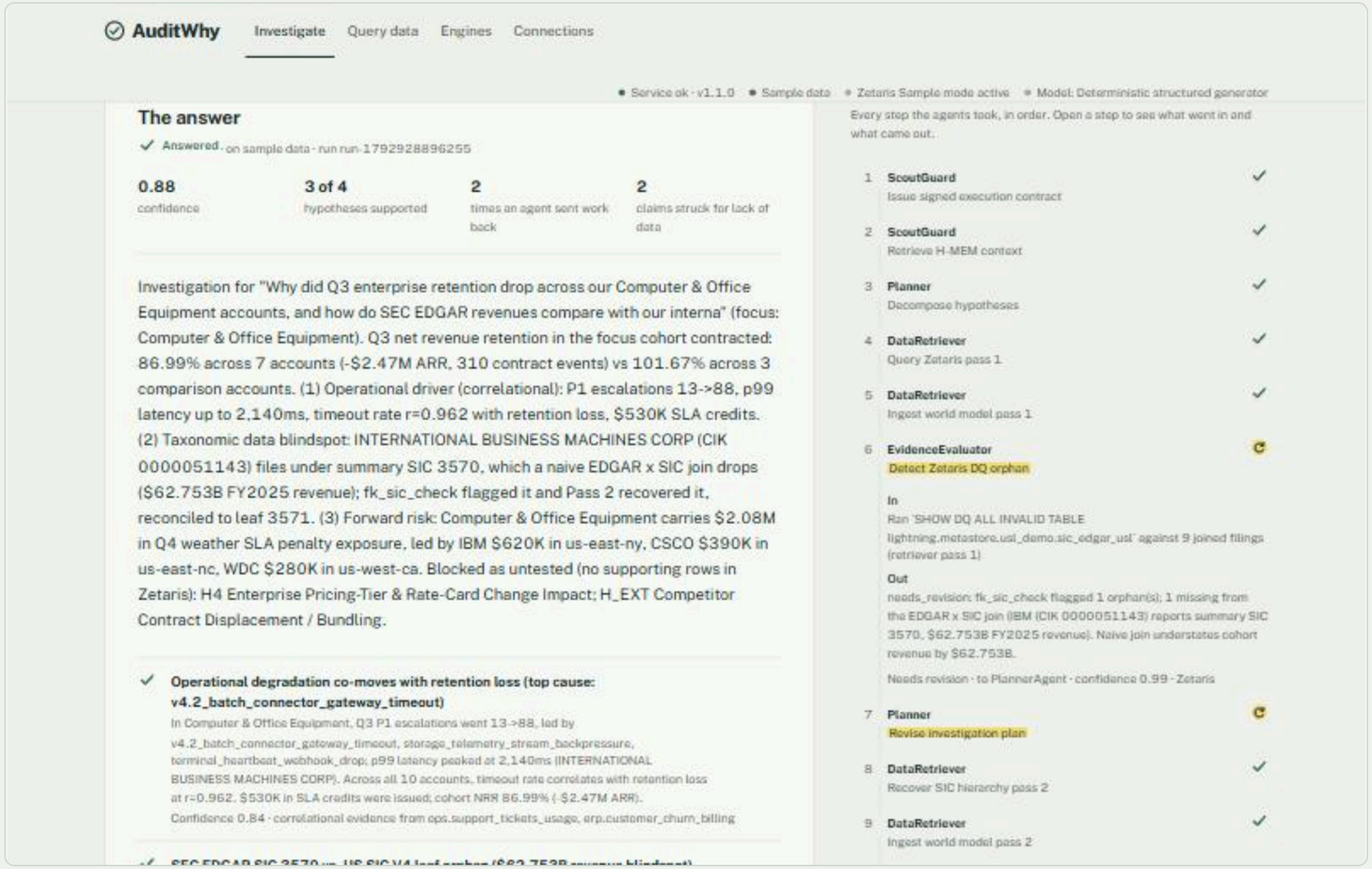
- All evidence read through Lightning SQL over REST
- Internal tables registered as REST sources and views
- Official sic_edgar_usl semantic layer joins EDGAR to SIC
- Its foreign-key DQ rule drives the revision loop

Remove Zetaris and there is no evidence and no verdict.

Meterless

- Scout signs a contract every Zetaris call verifies
- World Model merges three sources into stable entities
- Markovian bounds context; savings measured, not projected
- H-MEM stores rules with provenance and trust scores
- Nightshift serves every model call, routed to Nemotron

The product: an answer you can audit



The answer

Verdict, findings with tickmarks, struck untested claims, caveat and next steps.

How it got there

Every agent step in order. Revisions are highlighted, and each opens to show what went in and out.

Evidence ledger

Every Zetaris statement, the hypothesis it served, and its row count.

auditwhy.vercel.app

What comes next

Close the untested gaps

Mount pricing and CRM win/loss sources in Zetaris so H4 and competitor displacement become testable.

More taxonomies

Apply the same reconciliation to EU NACE and Japanese industry codes, which Zetaris already serves.

Memory that proposes rules

Promote H-MEM’s dream proposals into USL join rules after review, so fixes land at the source.

Scheduled audits

Run investigations each quarter-close and alert when a new orphan or contradiction appears.

github.com/Jae3y/AuditWhy · auditwhy.vercel.app